

S.G.Ganesh



# Why Care About Design Smells?

Design smells are poor solutions to recurring implementation and design problems. This article gives readers an overview on what design smells are, why we should be concerned about them, and what we can do to fix them.

**M**artin Fowler describes design smells as “structures in the code that suggest (sometimes they scream for) the possibility of refactoring.” Design smells are known by various other names, such as code smells, design flaws, design anti-patterns, etc. Martin Fowler popularised the term ‘smells’ in his book on refactoring, and this is the term known to most programmers today.

Why should we care about design smells? Because they are ‘problem patterns’ that can negatively impact the quality of the design, and ultimately affect the quality of the software. One way of looking at the cost of taking short-cuts to address immediate concerns is that the project accumulates ‘technical debt’. In other words, when we take short-cuts and don’t do the right thing, we have to pay a price for it—a form of ‘debt’. Over a period of time, if not addressed, this debt accumulates and when it crosses a threshold, it can result in completely derailing the software project. The stories of such failed projects are all too common. Note that this phenomenon is common in both commercial and open source projects.

Consider a software development project that was led by an inexperienced designer. Assume that the project was delivered under tight schedules, and the main objective of the project was to satisfy all the functional requirements elicited from the customers, without providing much focus on Non-Functional Requirements (NFRs). This short description of the project is in fact sufficient for you to understand that the code will have numerous smells! Now, let’s explore why.

One cause of design smells is when the designer makes wrong design decisions—a mistake that inexperienced designers are prone to make. Another cause is tight development schedules. When a project is developed under stiff and implausible deadlines, programmers and designers take numerous ‘short-cuts’, i.e., they do what works instead of doing what is right. From the code, you can observe these short-cuts in the form of design smells. When functionality completion is the main focus of software development and when NFRs are not given due importance, it results in sloppy design, which gets noticed as design smells.

Let us discuss a specific smell to get a better understanding of the subject.

One of the smells described by Martin Fowler is the ‘refused bequest’. The word ‘bequest’ means ‘a gift of personal property by will’. So, the term ‘refused bequest’

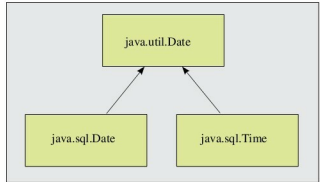


Figure 1: An example of the ‘refused bequest’ smell from JDK

means ‘not accepting inherited property’. In the context of object-oriented programming, a refused bequest means ‘derived classes reject what is inherited from the base class’.

Why is it a problem? Because it violates Liskov’s Substitution Principle (LSP), which is a cardinal rule in OO programming/design. LSP states that, “We should be able to use a base reference/pointer without being aware of whether it’s pointing to a derived object or not.” When there is no logical or conceptual ‘is-a relationship’ shared between the base and derived classes, we cannot use a base pointer/reference without knowing if it is pointing to the base type object or the derived type object.

Let us look at an example of this smell from the open source Java Development Kit (JDK). The `java.util.Date` class supports both date and time functionality. The class has two derived classes: `java.sql.Date` and `java.sql.Time`, which represent date and time responsibilities (see Figure 1). The `java.sql.Date` class supports only date-related functionality, and throws exceptions if time-related functionality is invoked! Consider the following code segment, for example:

```
java.util.Date date1 = new java.util.Date(2013, 01, 01);
System.out.println(date1.getSeconds());
// prints 0 since zero seconds elapsed
// from the start of the day
```

```
java.util.Date date2 = new java.sql.Date(2013, 01, 01);
System.out.println(date2.getSeconds());
// throws java.lang.IllegalArgumentException
// since time related methods
// are not supported in java.sql.Date
```